Tutorato 1

Matteo Mazzaretto

20 Ottobre 2026



Esempio di classe

```
class Triangle{
  private:
    double a, b, c;
  public:
    Triangle(double a_in, double b_in, double c_in);
    void setTriangle(double x, double y, double z);
};
```

Ogni metodo possiede un parametro implicito **this* che indica l'oggetto che invoca il metodo (oggetto di invocazione). Fatta eccezione per un metodo segnato con la keyword *static*.

La classe è un ADT (tipo di dato astratto): definisce un dato e le operazioni che si possono fare su di esso.

Namespace

I namespace vengono utilizzati per identificare una collezione di nomi ed identificatori appartenenti ad una stessa categoria.
Il più classico degli esempi è *std*.
Per accedere ai membri di un namespace, si usa l'operatore di scoping `::`

```
#include <iostream>

std::cout << "Hello World" << std::endl;
```

Modularizzazione

Per la definizione di ogni classe, servono due file: *classe.h* e *classe.cpp*
Il primo dichiara i campi e i metodi della classe; il secondo definisce i metodi, evitando la dichiarazione *inline*.

```
#ifndef TRIANGLE_H
#define TRIANGLE_H
class Triangle{
  private:
    double a, b, c;
  public:
    Triangle(double a_in, double b_in, double
c_in);
    Triangle& setA(double x);
    Triangle& setB(double y);
    Triangle& setC(double z);
};
#endif
```

```
//costruttore
Triangle::Triangle(double a_in, double b_in,
double c_in): a(a_in), b(b_in), c(c_in) {}

Triangle& Triangle::setA(double x) { a = x; return
*this; }
Triangle& Triangle::setB(double y) { b = y; return
*this; }
Triangle& Triangle::setC(double z) { c = z; return
*this;}
```

Costruttori

Come visto nell'esempio precedente, il costruttore è il metodo che permette di costruire un oggetto di quella classe.
I costruttori possono essere utilizzati per:

1. convertire tipi (nel caso precedente converte tre double in un oggetto di classe Triangle)
2. assegnare valori a seconda del numero di parametri (**overloading**)

Si può considerare pure il caso dei parametri di default:

```
class Triangle{
  private: double a, b, c;
  public: Triangle(double a_in, double b_in, double c_in = 4);
};
```

Costruttori

Ci sono 2 modi principali per scrivere un costruttore

1. Senza lista d'inizializzazione
2. Con lista d'inizializzazione

```
Triangle(double a_in, double b_in, double c_in) {
  a = a_in;
  b = b_in;
  c = c_in;
}
```

```
Triangle(double a_in, double b_in, double c_in) : a(a_in), b(b_in), c(c_in) {}
```

La keyword "const"

Serve per ottenere l'immutabilità e impedire che dati che non devono essere modificati vengano toccati per sbaglio.
Può essere utilizzata in due modi:

1. Sui metodi, quando non modificano i campi dell'oggetto d'invocazione
2. Sui parametri

```
class Triangle{
  private: double a, b, c;
  public:
    double perimeter() const { return a + b + c; }  // non modifica a, b, c
    bool isBigger(const Triangle& t) const;         // non modifica t
};
```

Passaggio per riferimento

Quando si costruisce una funzione, si può decidere se passare i parametri per valore (con copia) o per riferimento. In generale è meglio la seconda opzione poiché evita la copia e riduce il tempo di esecuzione; la copia per valore può essere inutile.

```
bool byValue(C x) {return true;}
bool byConstRef(const C& x) {return true;}
```

Costruttore di copia

Casi di invocazione:

1. Quando un oggetto viene dichiarato ed inizializzato con un altro oggetto della stessa classe
2. Quando un oggetto viene passato per valore come parametro di una funzione
3. Quando una funzione ritorna per valore tramite return un oggetto

Per fare la ridefinizione del costruttore di copia, si fa così:

```
class Triangle{
  private: double a, b, c;
  public:
    Triangle(const Triangle& t){
      a = t.a;
      b = t.b;
      c = t.c;
    }
};
```

Costruzione di oggetti

Nel codice, si può costruire un oggetto in più modi:

```
Triangle def; //costruttore di default

Triangle triangolo(5, 4, 7);

Triangle copia = triangolo;

Triangle not_all_default(5, 4); //avrà parametro c = 4

Triangle copia2(def);
```

Se si vuole evitare la conversione implicita, si può usare la keyword `explicit` davanti al costruttore.

Distruttore

Il distruttore viene invocato quando un oggetto deve essere cancellato dalla memoria:
I motivi principali sono i seguenti:

1. Chiusura del programma
2. Chiusura della funzione nella quale l'oggetto era stato costruito
3. Eliminazione dell'oggetto nel codice

Viene definito così:

```
class Triangle{
  public:
    ~Triangle();
};
```

Esercizio 1: Cosa stampa?

```cpp
#include <iostream>
using namespace std;

class Foo {
  int x;
public:
  Foo(int v = 10) : x(v) { cout << "C" << x << " "; }
  Foo(const Foo& f) : x(f.x + 1) { cout << "K" << x << " "; }
  ~Foo() { cout << "D" << x << " "; }
};

int main() {
  Foo a;
  Foo b(5);
  Foo c = a;
  return 0;
}
```

Cosa stampa a schermo la funzione `main()`?

Soluzione Esercizio 1

```
Output: C10 C5 K11 D11 D5 D10
```

Spiegazione:

1. `Foo a;` invoca il costruttore di default (`x=10`), stampa `C10`.
2. `Foo b(5);` invoca il costruttore ad un argomento (`x=5`), stampa `C5`.
3. `Foo c = a;` invoca il costruttore di copia, con `x = a.x + 1` (`11`), stampa `K11`.
4. Alla fine del `main`, gli oggetti vengono distrutti in ordine inverso di creazione: `c` (stampa `D11`), `b` (stampa `D5`), `a` (stampa `D10`).

Esercizio 2: Completamento codice

Completa i campi ed i modificatori mancanti (`/* 1 */`, `/* 2 */`, `/* 3 */`) affinché il codice sia corretto e sicuro rispetto ai principi di incapsulamento e immutabilità:

```
class Counter {
  private:
    int val;
  public:
    Counter(int v = 0) : val(v) {}

    // Deve ritornare true se val è maggiore di quello di c
    bool isGreater( /* 1 */ Counter& c ) /* 2 */;

    // Incrementa val e ritorna l'oggetto corrente per riferimento
    Counter& increment() /* 3 */;
};
```

Soluzione Esercizio 2

```
class Counter {
  private:
    int val;
  public:
    Counter(int v = 0) : val(v) {}

    /* 1 */ const
    /* 2 */ const
    /* 3 */ (nessuna keyword const!)
};
```

Spiegazione:

- `/* 1 */`: `const Counter& c` evita modifiche indesiderate all'oggetto passato per riferimento.
- `/* 2 */`: `const` rende il metodo leggibile anche su istanze costanti di `Counter`.
- `/* 3 */`: `increment()` modifica lo stato interno (`val`), quindi non deve essere `const`.

Ridefinizione degli operatori

Gli operatori di assegnazione (=, +=, -=, *=, /=) ritornano per riferimento per permettere la concatenazione. Anche [] ritorna per riferimento, per poter modificare l'elemento; == invece ritorna un `bool`.

```
class Triangle{
  private: double a, b, c;
  public:
    Triangle& operator=(const Triangle& t){
      a = t.a;
      b = t.b;
      c = t.c;
      return *this;
    }
};
```

Definizioni profonde

Quando una classe ha campi puntatore a memoria dinamica, solitamente:

- distruttore
- costruttore di copia
- assegnazione

devono essere ridefiniti, per impedire la condivisione di memoria fra due oggetti diversi.
La “regola del tre” dice che se una classe ha bisogno di ridefinire uno dei precedenti, allora con molta probabilità deve ridefinire anche gli altri 2.

Esempio: classe con puntatori

Consideriamo la classe Instagram, che contiene una lista di Reel:

```
class Instagram{
  private:
    class Reel{
      private:
        int sec;
      public:
        Reel* next;
        Reel(int s, Reel* n): sec(s), next(n) {}
    };
    Reel* first;
    Reel* copy(Reel* r);
    void destroy(Reel* r);
  public:
    Instagram(): first(nullptr) {}
    Instagram(const Instagram& x);
    Instagram& operator=(const Instagram& x);
    ~Instagram();
};
```

La regola del tre in azione

UNIVERSITÀ DI PADOVA

```
Instagram::Reel* Instagram::copy(Reel* r){
  if(r == nullptr)
    return nullptr;
  return new Reel(r->sec, copy(r->next));
}

void Instagram::destroy(Reel* r){
  if(r != nullptr){
    destroy(r->next);
    delete r;
  }
}

Instagram::Instagram(const Instagram& x): first(copy(x.first)) {}

Instagram& Instagram::operator=(const Instagram& x){
  if(this != &x){       // evita l'autoassegnazione
    destroy(first);     // libera la vecchia memoria
    first = copy(x.first);
  }
  return *this;
}

Instagram::~Instagram(){ destroy(first); }
```

Esercizio 3: Cosa stampa?

```cpp
#include <iostream>
using namespace std;

class Test {
public:
  Test() { cout << "Def "; }
  Test(const Test&) { cout << "Copy "; }
  Test& operator=(const Test&) { cout << "Assign "; return *this; }
};

void g(Test t) {}

int main() {
  Test t1;
  Test t2 = t1;
  Test t3;
  t3 = t1;
  g(t1);
  return 0;
}
```

Cosa stampa a schermo l'esecuzione del `main()`?

Soluzione Esercizio 3

```
Output: Def Copy Def Assign Copy
```

Spiegazione:

1. `Test t1;` -> `Def` (costruttore di default).
2. `Test t2 = t1;` -> `Copy` (è un'inizializzazione, non un'assegnazione!).
3. `Test t3;` -> `Def`.
4. `t3 = t1;` -> `Assign` (assegnazione tra oggetti già esistenti).
5. `g(t1);` -> `Copy` (passaggio di parametro per valore).

Grazie per l’attenzione!